The four projects

PROJECT	TITLE	COVERS	TOPICS	BY THE END
Project 1	Plan It and Write the Swift	Modules 0 to 2	16	An app that runs on iPhone and iPad and shows a summary of sample visits worked out by your own Swift functions, plus a written plan for the whole app.
Project 2	Model the Data and Build the Screens	Modules 3 to 4	17	All four tabs work with sample data. People can add, edit and delete places and visits until they quit the app.
Project 3	Design It Well and Save Everything	Modules 5 to 6	18	The app looks right in light and dark mode, at large text sizes and with VoiceOver, and everything people enter survives quitting the app.
Project 4	Connect It to the World and Ship It	Modules 7 to 9	21	A complete, tested app with a release plan, ready for TestFlight.

PROJECT 1 · MODULES 0 TO 2 · 16 TOPICS

Project 1: Plan It and Write the Swift

Create the Xcode project and the GitHub repository, plan the app, and write the Swift code that does the journal's math. A first simple screen shows the results for some sample places and visits.

> **By the end:** An app that runs on iPhone and iPad and shows a summary of sample visits worked out by your own Swift functions, plus a written plan for the whole app.

Set up

1. **Install Xcode.** Install **Xcode 26 or later** from the Mac App Store. Open it once, accept the license, and let it download the **iOS platform** (the Simulator and the iOS SDK). If the App Store says your macOS is too old, update macOS first.
2. **Sign in.** In **Xcode › Settings › Accounts**, click **+** and add your Apple Account. A free account gives you a *Personal Team*, which is enough to run on the Simulator and on your own iPhone. TestFlight and the App Store need the paid Apple Developer Program, which this project does not require.
3. **Create the project. File › New › Project › iOS › App**. Product Name: your app's name. Team: your Personal Team. Organization Identifier: reverse-DNS such as `com.yourname`. Interface: **SwiftUI**. Language: **Swift**. Testing System: **Swift Testing with XCTest UI Tests**. Storage: **None** (you add SwiftData yourself in 6.5). On the save panel, tick **Create Git repository on my Mac**.
4. **Set the build settings.** Select the project, then the app target. Under **General**, keep the minimum deployment at **iOS 26.0** and make sure both iPhone and iPad are supported. Under **Build Settings**, set **Swift Language Version** to **Swift 6** and leave **Default Actor Isolation** at **MainActor**.
5. **Organize by feature.** Create groups (folders) for `App`, `Models`, `Features/Places`, `Features/Map`, `Features/Insights`, `Features/Settings`, `Services` and `Resources`. Keep tests in the two test targets Xcode made.
6. **Put it on GitHub.** Create an **empty** repository on github.com (no README, no .gitignore). In Xcode's Source Control navigator (**⌘2**), right-click **Remotes › Add Existing Remote** and paste the repository URL, then **Integrate › Push**. Or, in Terminal inside the project folder: `git remote add origin <URL>` then `git push -u origin main`. Add a `.gitignore` with `xcuserdata/`, `DerivedData/` and `.DS_Store`.
7. **Commit as you go.** Commit after every requirement or two (**Integrate › Commit**, **⌥⌘C**) with a message that says what changed, and push at the end of each session. Your history shows the app growing; requirement 1.5 links to it.

Topics to cover

TOPIC	COURSE LESSON AND WHAT YOUR APP MUST DO
Module 0 · Start Here	
0.1	**Welcome to iOS Development** Create the Xcode project, put it on GitHub, and replace "Hello, world!" with a first screen that shows your app's name.
0.2	**Three Ways Swift Runs Here** Try one of your Swift functions in the course page's console before you add it to Xcode. Paste the experiment and what it printed into your README.

TOPIC	COURSE LESSON AND WHAT YOUR APP MUST DO
Module 1 · The Apple Platform and Its Ecosystem	
1.1	**One Swift Codebase, Many Screens** The app runs on iPhone and iPad: nothing is cut off on the smallest iPhone and nothing is stretched on an iPad.
1.2	**Xcode and the Developer Toolbox** Run the app on at least two simulators, a small iPhone and an iPad, and add a screenshot of each to your README.
1.3	**Anatomy of a SwiftUI App** In your README, name your `@main` App type, its `WindowGroup` scene and the root view, and say what each file Xcode created is for.
1.4	**The App Lifecycle** Read `scenePhase` and print a message each time the app becomes active, becomes inactive or moves to the background.
1.5	**The Software Development Life Cycle** Write a plan in your README for each phase of the life cycle: requirements (at least six user stories), design (sketches of three screens), implementation, testing, deployment and maintenance.
1.6	**Kinds of Apps and Extensions** In your README, name the kind of app you are building and one extension (a widget, a shortcut or a notification) that would help the people who use it, and why.
1.7	**Swift Beyond the iPhone** In your README, name another Apple platform your app could run on and which parts of your code could move there unchanged.

TOPIC	COURSE LESSON AND WHAT YOUR APP MUST DO
Module 2 · Swift Fundamentals	
2.1	**Values, Types and Your First Program** Use `let` unless a value changes, give every value a clear name, and keep the build at zero warnings.
2.2	**Numbers, Operators and Strings** Format numbers and dates for people with `formatted()`, and build a summary sentence with string interpolation, such as "12 walks since May".
2.3	**Optionals: Values That Might Be Missing** Values that can be missing, such as a visit's notes, are optionals unwrapped with `if let`, `guard let` or `??`. No force unwraps (`!`).
2.4	**Arrays, Dictionaries and Sets** Keep the sample visits in an array, count visits per place in a dictionary, and keep unique tags in a set.
2.5	**Decisions and Loops** Use `switch` to turn a rating or a weather code into words, `guard` for early exits, and at least one loop.
2.6	**Functions** Write the journal's calculations as small functions with clear argument labels, including one with a default parameter value and one that throws.
2.7	**Closures and Higher-Order Functions** Work out the summary with `filter`, `map`, `sorted(by:)` and `reduce`: total visits, average rating and the favorite place.

Hand in

- Push your work to the same GitHub repository you created in Project 1.
- Add a **Topics** section to your README with one row for each topic in this project: the topic number, and the file and type or function where your app does it, or the README section that answers it.
- Submit your GitHub repo link.

PROJECT 2 · MODULES 3 TO 4 · 17 TOPICS

Project 2: Model the Data and Build the Screens

Turn the sample data into real Swift types and build every screen in SwiftUI: the list of places, a place's details, the forms, Insights and Settings. The data lives in memory for now; Project 3 saves it.

> **By the end:** All four tabs work with sample data. People can add, edit and delete places and visits until they quit the app.

Topics to cover

TOPIC	COURSE LESSON AND WHAT YOUR APP MUST DO
Module 3 · Types, Protocols and Programming Paradigms	
3.1	**Structs: Your Own Value Types** Model values that get copied, such as a draft of a new visit, as structs. At least one conforms to `Equatable` or `Hashable`.
3.2	**Classes and Reference Semantics** Use a class where one shared instance matters, such as the model that holds all places. In your README, say why each of your types is a struct, class, enum or actor.
3.3	**Enums: Modeling Choices and State** A category enum (`String` raw values, `CaseIterable`, an SF Symbol name for each case) and a `LoadState` enum with associated values: idle, loading, loaded and failed.
3.4	**Protocols and Protocol-Oriented Design** A protocol for where places come from (for example `PlaceSource`) with a sample-data type that conforms to it. Screens depend on the protocol, not the type.
3.5	**Generics: Write It Once, Keep the Types** Write one generic type or function, such as `LoadState<Value>`, and use it with two different types.
3.6	**Error Handling** An error enum that conforms to `LocalizedError`, thrown and caught with `do`-`catch`, with its message shown to the person.
3.7	**Concurrency: async/await and Actors** Load the sample data with `async`/`await` (a short `Task.sleep` stands in for the network), build in the Swift 6 language mode with no warnings, and write one `actor`.
3.8	**Programming Paradigms in Swift** In your README, point to one place in your code for each paradigm: object-oriented, protocol-oriented, functional, declarative and reactive.

TOPIC	COURSE LESSON AND WHAT YOUR APP MUST DO
Module 4 · SwiftUI and Reactive Interfaces	
4.1	**Describe the Screen, Not the Steps** Build every screen in SwiftUI. In your README, explain which state change redraws which part of one screen.
4.2	**Views and Modifiers** Write at least four reusable views of your own that take parameters (such as `PlaceRow` and `RatingView`) and one custom view modifier.
4.3	**Stacks, Frames, and Grids** Lay out screens with stacks, spacing and alignment, and show something in a `LazyVGrid`, such as the categories.
4.4	**State Drives the Screen** Use `@State` for values one view owns, and build a rating control that edits its parent's value through a `@Binding`.
4.5	**Observable Models** One `@Observable` model shared by several screens through the environment.
4.6	**Lists and Dynamic Data** A list of places with sections, swipe to delete, search, and an empty state (`ContentUnavailableView`).
4.7	**Navigation, Tabs, and Sheets** A `TabView` with four tabs (Places, Map, Insights, Settings), a `NavigationStack` into a place's details, and a sheet for adding. The Map tab can be a placeholder until Project 4.
4.8	**Forms and Input** Add and edit forms with a text field, a picker, a date picker, a toggle and your rating control. Save stays disabled until the input is valid.
4.9	**Motion, Adaptivity, and Liquid Glass** At least one animation that respects Reduce Motion, and one layout that adapts with `ViewThatFits` or the size class.

Hand in

- Push your work to the same GitHub repository you created in Project 1.
- Add a **Topics** section to your README with one row for each topic in this project: the topic number, and the file and type or function where your app does it, or the README section that answers it.
- Submit your GitHub repo link.

PROJECT 3 · MODULES 5 TO 6 · 18 TOPICS

Project 3: Design It Well and Save Everything

Polish the design with Apple's Human Interface Guidelines, then make the data last: settings, an export file, a SwiftData database and a passcode in the Keychain.

> **By the end:** The app looks right in light and dark mode, at large text sizes and with VoiceOver, and everything people enter survives quitting the app.

Topics to cover

TOPIC	COURSE LESSON AND WHAT YOUR APP MUST DO
Module 5 · Designing Great iOS Apps	
5.1	**Principles of iOS Design** In your README, show three principles from the Human Interface Guidelines that your app follows, each with a screenshot.
5.2	**Liquid Glass** Use the standard bars and tab bar, and add one floating control of your own with the glass effect. Content underneath stays readable.
5.3	**Typography and Dynamic Type** Use text styles only, never fixed font sizes. Nothing is cut off at the largest accessibility text size.
5.4	**Color, Dark Mode, and Contrast** Use colors that work in light and dark mode with readable contrast, and never use color as the only signal.
5.5	**Layout, Spacing, and Touch Targets** Every tappable element is at least 44 by 44 points, spacing is consistent, and content stays inside the safe area.
5.6	**SF Symbols and App Icons** Use SF Symbols for every icon, and give the app an icon you designed.
5.7	**Choosing a Navigation Pattern** In your README, draw the screen flow and explain why tabs, stacks and sheets suit your app.
5.8	**States, Feedback, and Clear Copy** Every screen that loads or can be empty has empty, loading, error (with a Retry button) and content states. Every button says what it does.
5.9	**Designing for Accessibility** VoiceOver labels on icon buttons and images, list rows read as one element, and an adjustable rating control. In your README, note what you fixed after trying VoiceOver.
5.10	**Process, Prototypes, and Critique** Show the app to one person, write down their feedback, and make three changes because of it. Put before-and-after screenshots in your README.

TOPIC	COURSE LESSON AND WHAT YOUR APP MUST DO
Module 6 · Storing Data: Local Storage and Databases	
6.1	**Where App Data Lives** In your README, make a table of every piece of data your app keeps, where it lives, and why it lives there.
6.2	**Settings with @AppStorage** Store the units, the sort order and a show-once welcome screen with `@AppStorage`.
6.3	**Codable and JSON** An export type that encodes to JSON and decodes back with `Codable`. Prove the round trip by decoding what you encoded (use whole-second dates: `.iso8601` drops fractions of a second).
6.4	**Reading and Writing Files** Export writes the JSON to the app's Documents folder, shows a message if it fails, and offers the file with `ShareLink`.
6.5	**SwiftData Fundamentals** Places and visits are `@Model` classes in one model container, listed with `@Query`, added and deleted through the model context, and filtered with `#Predicate`.
6.6	**Relationships and Schema Changes** A place has many visits, deleting a place deletes its visits, and previews use an in-memory container with sample data.
6.7	**Secrets in the Keychain** Places can be marked private and hidden behind a passcode saved in the Keychain, never in `@AppStorage` or a file.
6.8	**iCloud and CloudKit Sync** Keep the models ready for iCloud (every property has a default or is optional, relationships are optional, no unique attributes), and explain in your README how syncing would work.

Hand in

- Push your work to the same GitHub repository you created in Project 1.
- Add a **Topics** section to your README with one row for each topic in this project: the topic number, and the file and type or function where your app does it, or the README section that answers it.
- Submit your GitHub repo link.

PROJECT 4 · MODULES 7 TO 9 · 21 TOPICS

Project 4: Connect It to the World and Ship It

Add location, the map, photos, live conditions from the internet, an on-device summary, a shortcut and a reminder. Then test, debug and prepare the app for release.

> **By the end:** A complete, tested app with a release plan, ready for TestFlight.

Topics to cover

TOPIC	COURSE LESSON AND WHAT YOUR APP MUST DO
Module 7 · Device APIs: Location, Maps, Cloud and More	
7.1	**Permission and Privacy** A plain-language location purpose string, a screen that explains why before the system asks, and a button that opens Settings if permission is denied.
7.2	**Core Location, the Modern Way** Fill a new place's coordinates from your location, show each place's distance from you, and suggest a name with `MKReverseGeocodingRequest` inside a `@concurrent` function.
7.3	**MapKit for SwiftUI** The Map tab shows a marker for every place and your own location, a button that fits all places on screen, and a search for adding a place.
7.4	**Cloud Storage with CloudKit** In your README, plan how your app would use CloudKit: which database, checking the iCloud account first, and handling errors. Building it needs the paid developer program.
7.5	**Photos, Camera and Sharing** Attach a photo to a visit with `PhotosPicker`, and share a place with `ShareLink`.
7.6	**On-Device Intelligence** A Summarize button uses the Foundation Models framework when the on-device model is available, and explains why not when it is unavailable.
7.7	**Beyond the App: Widgets, Intents and Notifications** An App Shortcut built with App Intents, and a reminder sent as a local notification.

TOPIC	COURSE LESSON AND WHAT YOUR APP MUST DO
Module 8 · Networking and Backend Communication	
8.1	**How an App Talks to a Server** Log the protocol and time of your conditions request with `URLSessionTaskMetrics`, and trace one request step by step in your README.
8.2	**Speaking HTTP** Build request URLs with `URLComponents`, and check the status code of every response.
8.3	**REST Resources and JSON** Decode your Open-Meteo response into Swift types with `CodingKeys`.
8.4	**URLSession with async/await** Load live conditions for a place with `URLSession` and `async` / `await`, started with `.task` so leaving the screen cancels it.
8.5	**Errors, Retries and Security** Friendly messages for network errors, up to three retries with growing delays for temporary failures, HTTPS only, and no secrets in the app.
8.6	**Live Data Beyond Request and Response** Refresh conditions on a schedule while a place is open, plus pull to refresh. In your README, explain why polling fits better than WebSockets or push for this data.
8.7	**Designing the API Layer** A protocol for the conditions service with a live version and a stub for previews and tests, and a configured `URLCache`.

TOPIC	COURSE LESSON AND WHAT YOUR APP MUST DO
Module 9 · Architecture, Testing and Shipping	
9.1	**Architecture That Survives Change** Use MVVM: views show state, view models hold the logic and receive their services through `init`, and folders are organized by feature.
9.2	**Testing with Swift Testing** At least 12 Swift Testing tests, including a parameterized test and view model tests that use the stub service.
9.3	**UI Tests, Previews and Accessibility Checks** One UI test that adds a place and finds it in the list, one accessibility audit test, and a `#Preview` for every main screen.
9.4	**Debugging and Instruments** Log with `Logger` instead of `print`. Keep a bug log in your README: two real bugs, how you found each one, and the fix.
9.5	**Performance, Accessibility and Localization** Make photo thumbnails off the main actor, record one Instruments session, and translate the app into a second language with a String Catalog.
9.6	**From Archive to App Store** Set the version and build number, add a privacy manifest, archive the app, and write a release plan in your README.
9.7	**Capstone: Study Buddy from Idea to Release** Turn the user stories from Project 1 into acceptance criteria, and link each criterion to the test that proves it.

Hand in

- Push your work to the same GitHub repository you created in Project 1.
- Add a **Topics** section to your README with one row for each topic in this project: the topic number, and the file and type or function where your app does it, or the README section that answers it.
- Submit your GitHub repo link.